

TECHNICAL REPORT

Field	Details
Institute / University	Pimpri Chinchwad College of Engineering (PCCOE), Pune
Programme / Batch	B.Tech Computer Science and Engineering (AI & ML) / 2027 Batch
Student Name	Rupam Gunavant Karale
PRN	123B1B164
Case Study ID (CS1–CS5)	CS4
Project Title	Secure Code Debugging and Review Assistant
GitHub Repository	https://github.com/Rupam1661/PCCOE_Rupam-Karale_123B1B164_CS4_CSE

1. Problem Statement and Intended Users

Embedded automotive software is mostly written in C, where defects such as unbounded copies, unchecked allocations and hard-coded credentials are easy to introduce and expensive to find late. Reviewers spend time reading code, matching it against coding rules, and writing up why a finding matters. The project provides an assistant that brings these activities into one controlled workflow: a deterministic scanner finds known-risky patterns, retrieval-augmented generation (RAG) explains them using the code and coding rules as evidence, and an engineer accepts or rejects each finding. The intended users are embedded and automotive software engineers, code reviewers, and students working with secure-coding checks who cannot send source code to an external cloud service.

2. Input Data / Knowledge Base

Item	Description
Sources	Synthetic content created for the project: a C source file (sample_code/ecu_can.c) modelled on a CAN-frame ECU module, and a coding-rules file (coding_rules.txt) with 8 synthetic internal rules. No proprietary standard or OEM code is used.
Size and format	ecu_can.c: about 60 lines, with 8 deliberately seeded defects and one correct function (safe_alloc) used as a negative control. coding_rules.txt: 8 rules, each with an ID, severity and instruction. Results are stored in a local SQLite database (findings.db) and exported as CSV and JSON.
Preprocessing / EDA done	Source is read as text and split into line windows; rules are read one per chunk. Because the knowledge base is source code and short rule text, numerical EDA is not applicable.
Chunking strategy	Code: 12-line windows overlapping by 4 lines, giving 7 chunks (1-12, 9-20, 17-28, 25-36, 33-44, 41-52, 49-60), each stored with file name and line range. Rules: one chunk per rule (8). Retrieval returns the top 3 code chunks and top 2 rule chunks.

Seeded defects, as listed by the deterministic scanner:

Line	Rule	Severity	Finding	Pattern in the code
6	SEC-03	high	Hard-coded credential	static const char *diag_password = "..."

Line	Rule	Severity	Finding	Pattern in the code
15	SEC-04	high	Possible format-string vulnerability	printf(user_msg)
19	BND-01	medium	memcpy length not clearly bounded	memcpy(f->data, src, n)
24	SEC-02	high	Unbounded string function	strcpy(dst, name)
28	MEM-01	medium	Allocation result not checked for NULL	malloc without a NULL check
34	RES-01	medium	fopen result not checked for NULL	fopen without a NULL check
42	SEC-01	critical	Use of gets()	gets(buf)
46	STY-01	low	Use of goto	goto fail

3. Solution Architecture

Component	Choice (name, version, hosting)
Pre-trained model	Ollama 0.40.0 running llama3.2:3b locally, temperature 0.1. Used for specification-grounded answers, finding explanations with a suggested fix, and log analysis. No external API.
Embedding model	Sentence Transformers all-MiniLM-L6-v2 (sentence-transformers 6.1.0), run locally. The model is downloaded once on first run.
Vector store (FAISS / ChromaDB)	FAISS (faiss-cpu 1.15.1): two local inner-product indexes, one for code chunks and one for coding rules.
RAG pipeline / AI-agent design	Question or finding → embed → retrieve top-3 code chunks and top-2 rules → prompt builder (answer from context only, code treated as untrusted, cite [FILE:lines] and [RULE:ID]) → local LLM → cited response. No autonomous agent; nothing is executed or changed automatically.
Service layer and UI	Streamlit (1.65.0) interface with four tabs: Code Q&A, Review scan, Log / warning analysis, Findings & export. Python modules handle scanning (analyzers.py), retrieval and storage. SQLite stores the review log.
Tools / deterministic rules	analyzers.py: 8 regular-expression rules with a look-ahead that checks for a NULL test after malloc and fopen. The scanner decides what is a finding; the LLM only explains it.

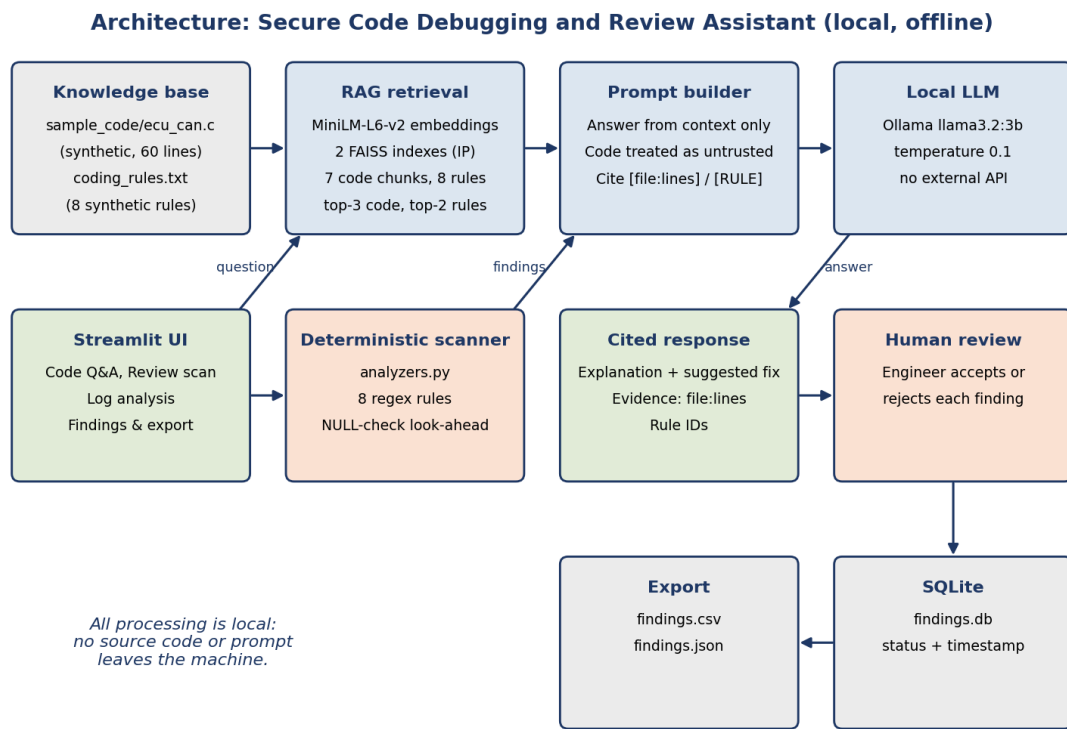


Figure 1: System architecture of the Secure Code Debugging and Review Assistant (all processing is local).

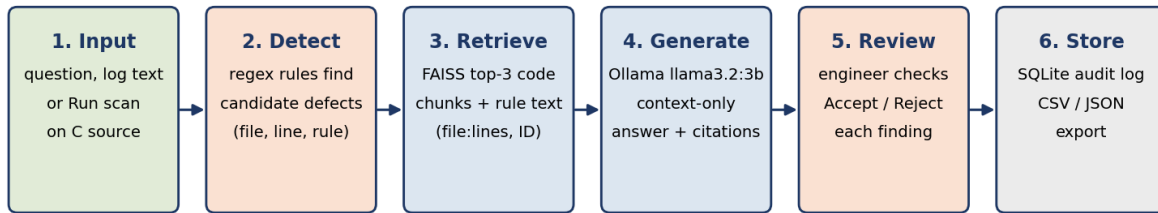
4. Implementation and Configuration

The implementation follows a simple flow: knowledge-base indexing → deterministic detection → retrieval → AI explanation → human review → storage and export.

At start-up the C file and the rules are chunked, embedded with all-MiniLM-L6-v2 and added to two FAISS indexes. The sidebar shows the number of code chunks (7), the number of rules (8), the model and the embedding model. In Code Q&A the engineer asks a question; the top code and rule chunks are retrieved and passed to llama3.2:3b, whose answer cites the file and line range and the rule ID. In Review scan the engineer runs the deterministic scan, selects a finding and clicks “Explain with AI”; the model explains the root cause, the risk and a fix using the retrieved evidence. The engineer then chooses “Accept finding” or “Reject (false positive)”, and the decision is saved to SQLite with a timestamp. In Log / warning analysis, a pasted compiler or static-analysis line is combined with retrieved code and rules to propose a root cause and fix. The Findings & export tab shows reviewed, accepted and rejected counts and exports the stored findings as CSV or JSON.

The prompt instructs the model to answer only from the supplied context and to treat the code under review as untrusted data, so comments or strings inside the code cannot act as instructions (prompt-injection mitigation). The run uses a Python virtual environment, and dependencies are listed in requirements.txt.

End-to-end workflow: input → retrieval → explanation → human review → export



Step 2 applies to Review scan; Q&A and log analysis go from step 1 to step 3.

AI output is advisory only: no auto-fix, no auto-merge, no code execution.

Figure 2: End-to-end workflow from input to detection, retrieval, explanation, review and storage.

5. Evaluation Steps and Results

Evaluation uses the 8 seeded defects as ground truth for the scanner, five retrieval questions with an expected function name, and manual review of sample AI outputs. The scripted checks are run with Code/run_eval.py, and its output is saved in Evaluation_Results/ (eval_summary.json and retrieval_eval.csv).

#	Test question / scenario	Expected answer / behaviour	Result (Pass/Fail, notes)
1	Run the deterministic scan on ecu_can.c	All 8 seeded defects found; no finding on safe_alloc	Pass – 8 of 8 detected, 0 false positives, 0 false negatives
2	Which function copies a name using strcpy?	set_name in the top-3 chunks	Pass – retrieved chunks 17-28, 9-20, 25-36
3	Where is the hard-coded password stored?	diag_password in the top-3 chunks	Pass – retrieved chunks 1-12, 33-44, 9-20
4	Which function reads input with gets and can overflow?	read_input in the top-3 chunks	Pass – retrieved chunks 41-52, 33-44, 25-36
5	Where is goto used for error handling?	retry in the top-3 chunks	Pass – retrieved chunks 41-52, 49-60, 33-44
6	Which function opens the config file with fopen?	read_config in the top-3 chunks	Pass – retrieved chunks 33-44, 25-36, 49-60
7	Code Q&A: “Which function is vulnerable to buffer overflow?”	Names copy_payload and cites the evidence	Pass – answer names copy_payload and cites [FILE:ecu_can.c:17-28] [RULE:BND-01]
8	Explain with AI on finding SEC-03 (hard-coded credential), then Accept	Root cause, risk and fix; decision stored	Pass – explanation cites SEC-03; finding saved as ACCEPTED with timestamp
9	Log analysis: “ecu_can.c:46: warning: implicit declaration of function ‘gets’”	Recommend replacing gets() with fgets() (SEC-01)	Partly correct – fgets() fix is right, but the root-cause sentence is imprecise (see Section 9)

Metric	Score / observation
Scanner precision / recall	1.0 / 1.0 (8 true positives, 0 false positives, 0 false negatives) on the seeded file.
Retrieval hit rate	1.0 (5 of 5 questions): the expected function was in the top-3 retrieved code chunks.

Metric	Score / observation
Answer relevance / faithfulness	Manual review of sample outputs (tests 7–9). No numeric faithfulness score was collected.
Citation accuracy	Answers cite file, line range and rule ID; the “Retrieved evidence / citations” panel shows the supporting chunks with similarity scores and the matched rules.
Latency	Retrieval time printed by run_eval.py: 0.008 s to 0.429 s per question. End-to-end LLM response time was not formally benchmarked; it depends on local inference.

```
(venv) (base) rupam@Rupams-MacBook_PCCOE_Rupam_PRN_CS4_AI&L % cd Code
python run_eval.py
SCANNER: {'seeded_bugs': 8, 'detected': 8, 'TP': 8, 'FP': 0, 'FN': 0, 'precision': 1.0, 'recall': 1.0}
modules.json: 100% | 349/349 [00:00:00.00, 914KB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
config_sentence_transformers.json: 100% | 116/116 [00:00:00.00, 520KB/s]
README.md: 100% | 10.5k/10.5k [00:00:00.00, 12.4MB/s]
sentence_bert_config.json: 100% | 53.0/53.0 [00:00:00.00, 51.2KB/s]
config.json: 100% | 612/612 [00:00:00.00, 1.20MB/s]
model.safetensors: downloading bytes: | 85.0MB, 5.22MB/s
model.safetensors: reconstructing file: 100% | 90.9MB / 90.9MB, 7.36MB/s
loading weights: 100% | 183/183 [00:00:00.00, 22462.521t/s]
tokenizer_config.json: 100% | 350/350 [00:00:00.00, 898KB/s]
vocab.txt: 100% | 232k/232k [00:00:00.00, 10.4MB/s]
tokenizer.json: 100% | 466k/466k [00:00:00.00, 19.0MB/s]
special_tokens_map.json: 100% | 112/112 [00:00:00.00, 259KB/s]
config.json: 100% | 190/190 [00:00:00.00, 479KB/s]
{'question': 'Which function copies a name using strcpy?', 'expected_keyword': 'set_name', 'hit_in_top3': True, 'cited': 'ecu_can.c:17-28; ecu_can.c:9-20; ecu_can.c:25-36', 'latency_s': 0.429}
{'question': 'Where is the hard-coded password stored?', 'expected_keyword': 'diag_password', 'hit_in_top3': True, 'cited': 'ecu_can.c:1-12; ecu_can.c:33-44; ecu_can.c:9-20', 'latency_s': 0.141}
{'question': 'Which function reads input with gets and can overflow?', 'expected_keyword': 'read_input', 'hit_in_top3': True, 'cited': 'ecu_can.c:41-52; ecu_can.c:33-44; ecu_can.c:25-36', 'latency_s': 0.145}
{'question': 'Where is goto used for error handling?', 'expected_keyword': 'retry', 'hit_in_top3': True, 'cited': 'ecu_can.c:41-52; ecu_can.c:49-60; ecu_can.c:33-44', 'latency_s': 0.008}
{'question': 'Which function opens the config file with fopen?', 'expected_keyword': 'read_config', 'hit_in_top3': True, 'cited': 'ecu_can.c:33-44; ecu_can.c:25-36; ecu_can.c:49-60', 'latency_s': 0.15}
RETRIEVAL HIT RATE: 1.0
```

Figure 3: run_eval.py output – scanner precision/recall and retrieval hit rate.

6. Responsible AI Measures

Area	Measure adopted in this project
Grounding and citations	The prompt restricts answers to retrieved code and rules and asks for [FILE:lines] and [RULE:ID] citations; the evidence panel lets the engineer check each source.
Privacy	The LLM, embeddings, vector index and database all run locally. No source code or prompt is sent to an external service after the one-time model downloads.
Security	AI output is advisory only: no automatic fix, merge or code execution. The code under review is treated as untrusted input in the prompt.
Human oversight / review	Every finding is accepted or rejected by an engineer and logged with a timestamp. AI explanations are treated as assistance, not as a verdict.
Data	Only synthetic code and rules are used; no personal, confidential or proprietary data.

7. Innovation Highlights

- Combines deterministic detection with specification-grounded RAG explanations, so the model never decides what counts as a finding.
- Runs fully offline with a small local model, suitable for code that cannot leave the machine.
- Provides a complete review flow: detect → retrieve → explain → accept or reject → store → export.
- Includes a negative-control function (safe_alloc) so false positives are measured, not assumed.

8. Screenshots, Logs and Evidence

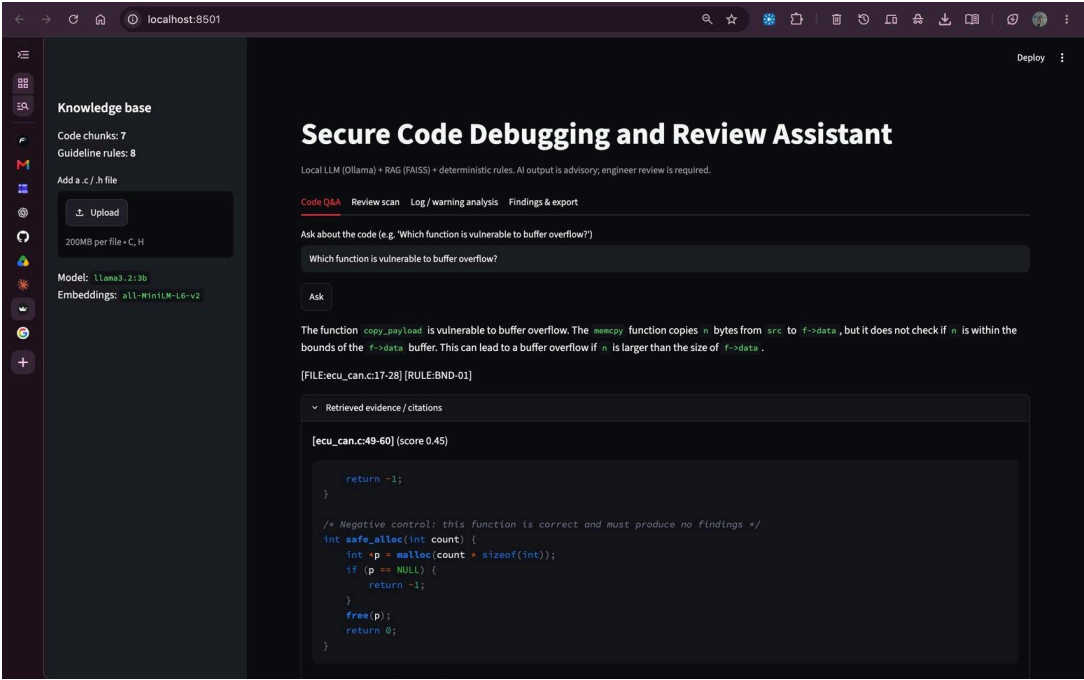


Figure 4: Code Q&A – answer with [FILE:lines] and [RULE:ID] citations.

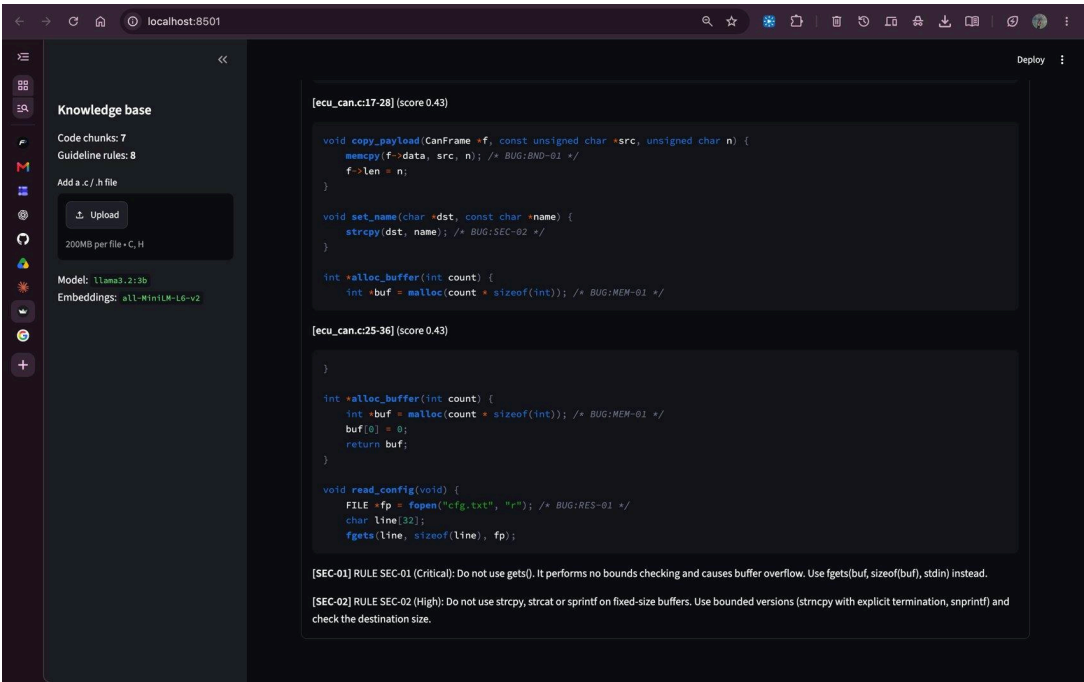


Figure 5: Retrieved evidence – code chunks with similarity scores and matched rules.

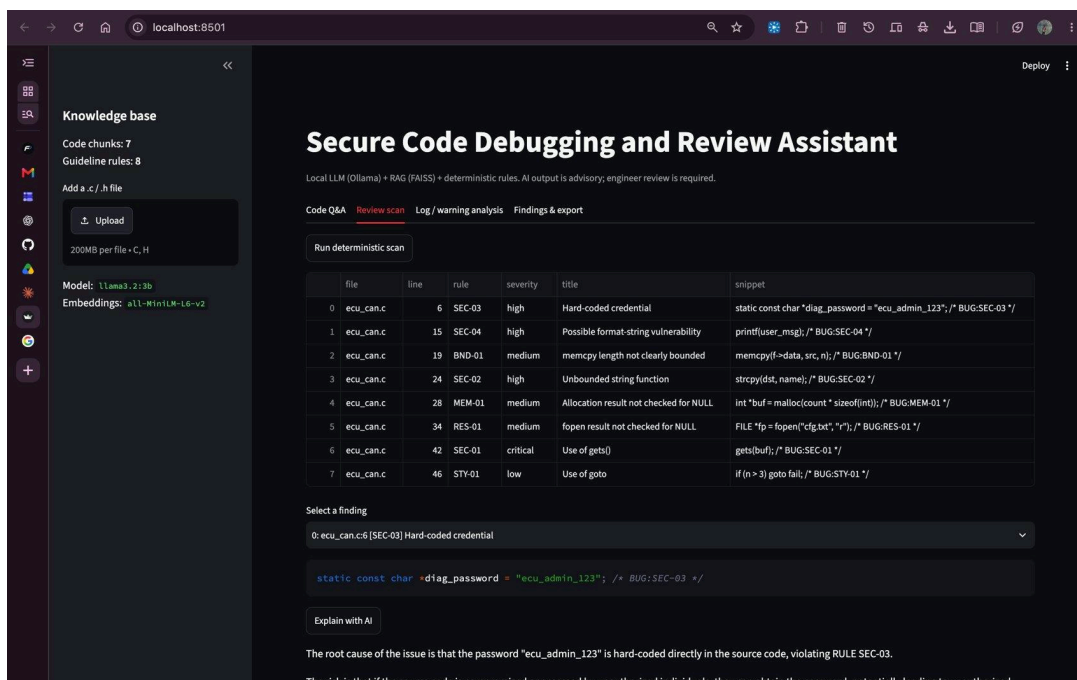


Figure 6: Review scan – 8 findings from the deterministic scanner; Explain with AI on SEC-03.

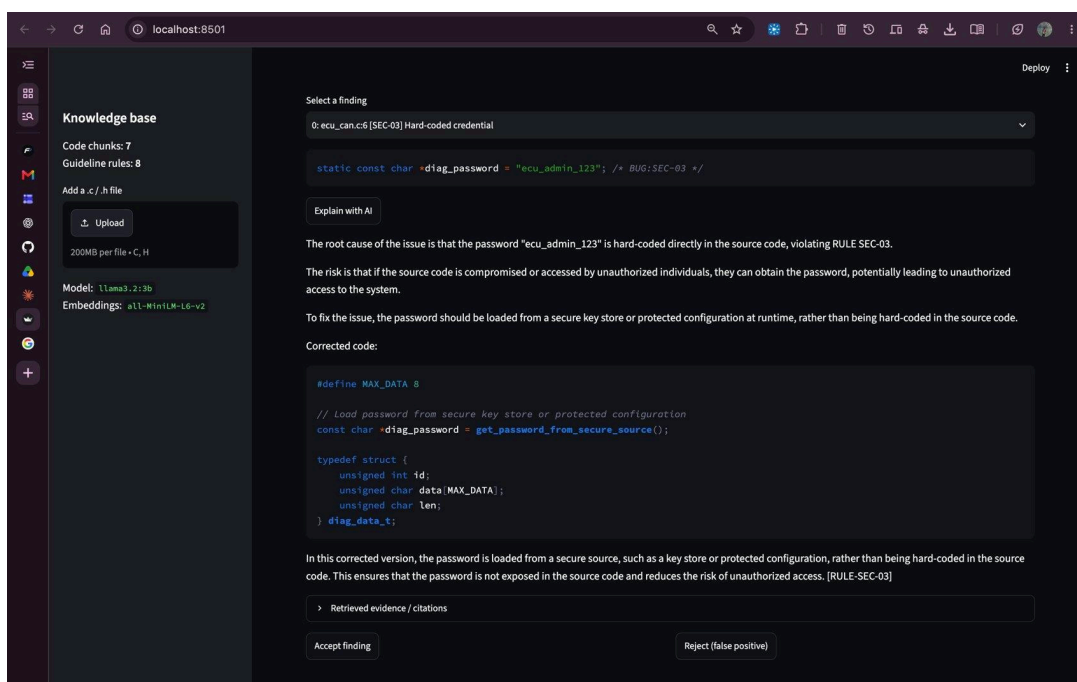


Figure 7: AI explanation and suggested fix, with Accept finding and Reject (false positive) buttons.

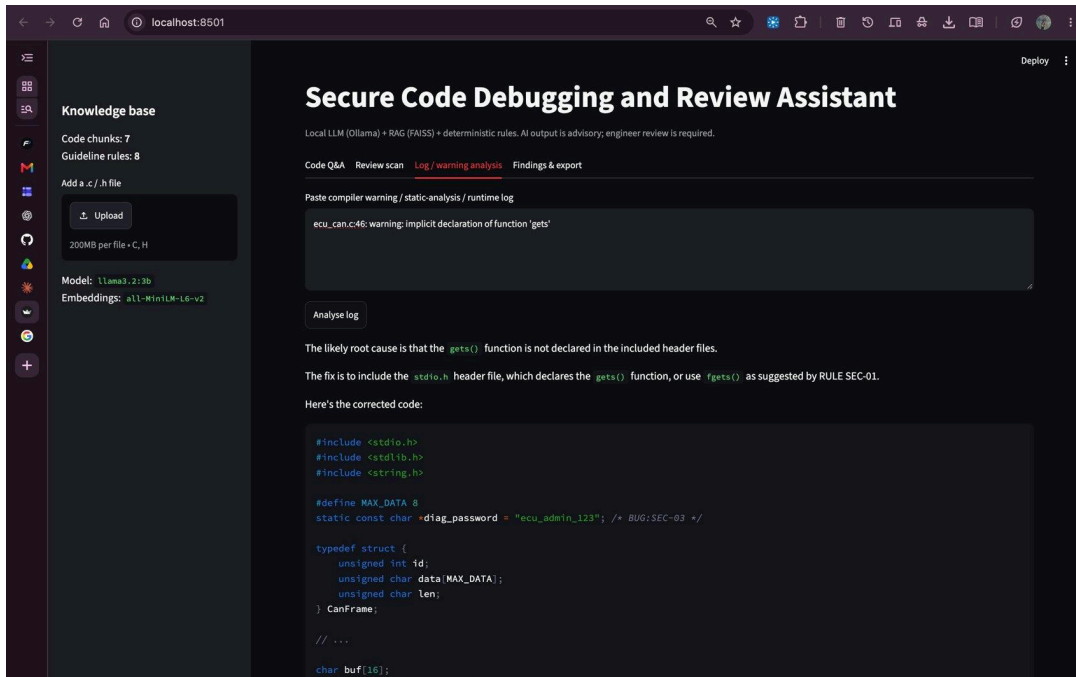


Figure 8: Log / warning analysis – input log line and AI analysis.

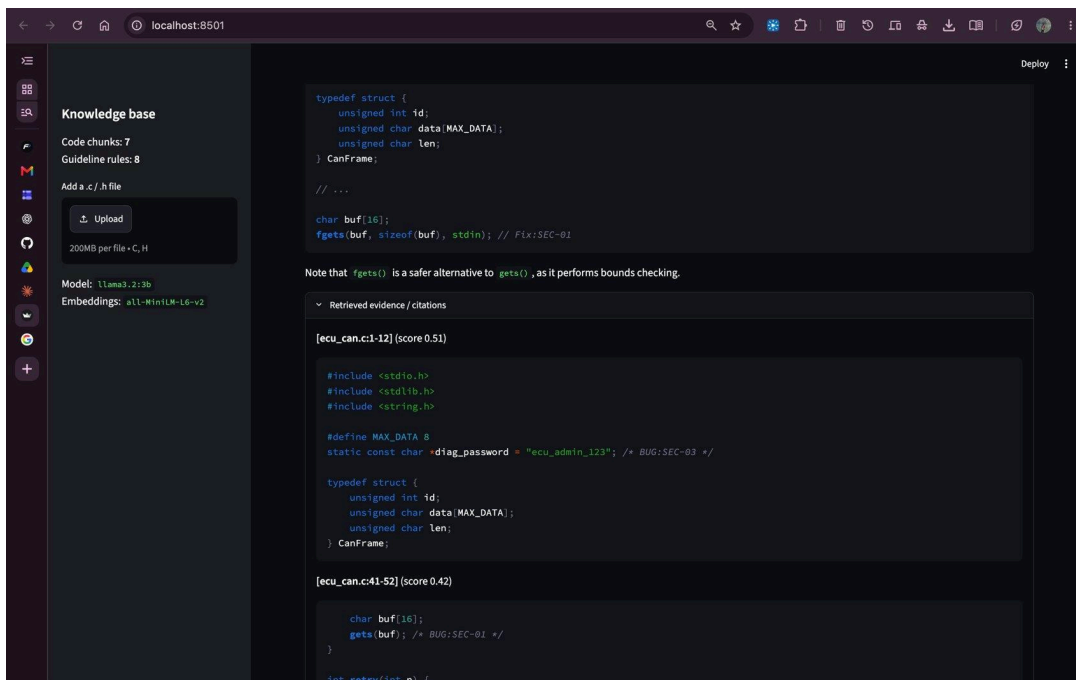


Figure 9: Log analysis – retrieved evidence (chunks 1-12 and 41-52).

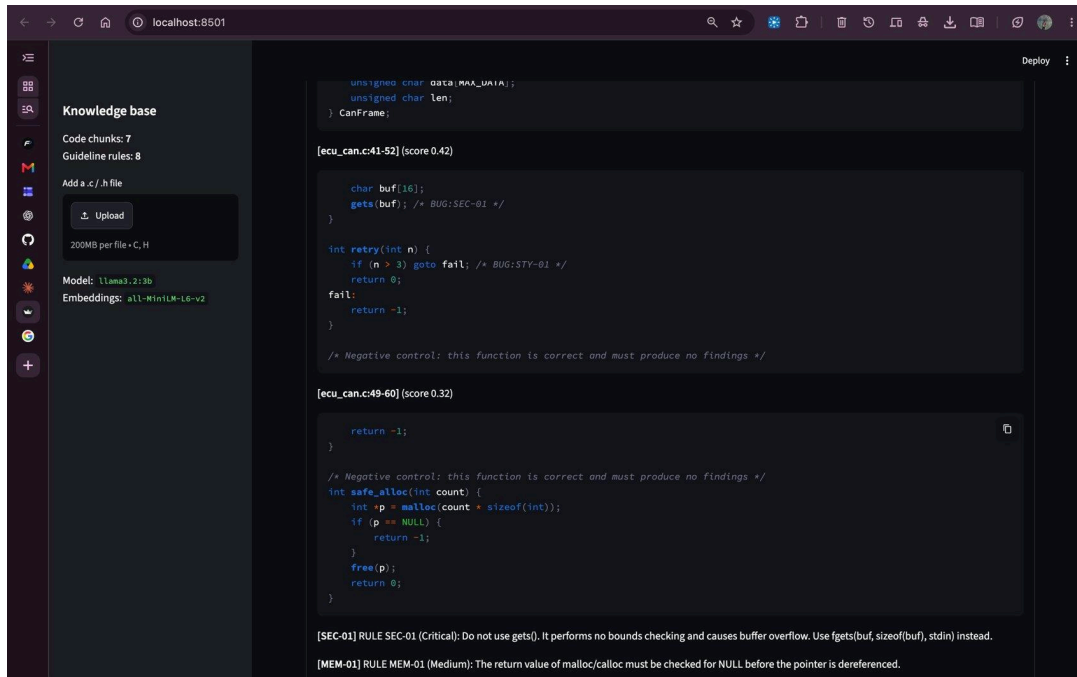


Figure 10: Log analysis – retrieved evidence continued, with rules SEC-01 and MEM-01.

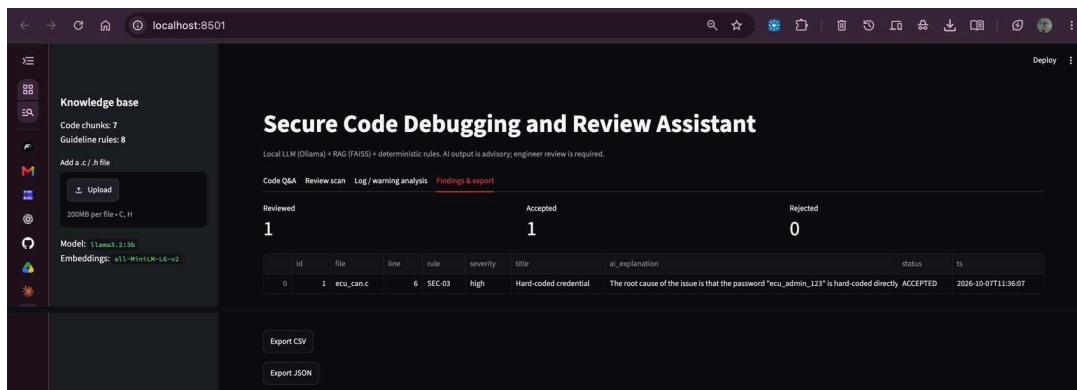


Figure 11: Findings & export – review counts, stored finding (ACCEPTED, with timestamp) and export buttons.

9. Reflection and Conclusions

The project shows a practical way to support secure code review with AI without handing the decision to the model. The deterministic scanner decides what is a finding, RAG grounds the explanation in the code and the coding rules, and the engineer makes the final accept or reject decision, which is stored and exportable. On the synthetic test file the scanner found all 8 seeded defects with no false positives, and retrieval returned the expected function for all 5 test questions.

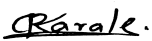
The results are limited. The data set is a single synthetic file of about 60 lines, and the scanner was tested on defects seeded by the author, so the scores of 1.0 show that the rules match the seeded patterns, not that they generalise to other code. With only 7 code chunks, returning the top 3 already covers a large part of the file, so the retrieval test is weak. The 3-billion-parameter model is also not always precise: in the log analysis example it stated that gets() was not declared in the included headers, although stdio.h is included at the top of the file, while its recommendation to use fgets() under rule SEC-01 was correct. This is why AI output is advisory and every finding is reviewed by an engineer. Natural next steps are testing on larger real open-source C code, adding a labelled question set with answer-correctness scoring, replacing regular expressions with a parser-based analyser, and supporting multi-file projects.

10. AI-Tool Usage Declaration

AI tool / model	Purpose	Artifact affected	Verified By Student (Y/N)	Remarks / Notes
Claude (Anthropic)	Coding assistance, generation of the synthetic C file and rules, debugging, and drafting of the synopsis and technical report.	Python modules, synthetic knowledge base, project documentation, synopsis and technical report.	Y	Student ran the code and evaluation on a local machine and reviewed the affected artifacts.
Ollama / llama3.2:3b	Local LLM used by the application for code Q&A, finding explanations and log analysis.	RAG/AI modules and generated explanation content.	Y	Student reviewed the model output; it can be imprecise and requires engineer review.

11. Student Declaration

I confirm that this report represents my own understanding and contribution, that the work follows the approved case-study scope, and that I can explain every significant part of the implementation.

Signature:		Date:	07-10-2026
Name:	Rupam Gunavant Karale	PRN:	123B1B164